SWEN30006
Software Design and Modelling

Review &
Exam Information

"Striving for success without hard work is like trying to harvest where you haven't planted."

—David Bly

Disclaimer

- Today's lecture will summarise some key topics taught in this subject
- This slide may ***NOT*** cover all the important details related to the key topics
- You should ***NOT*** solely use this review for a review for your exam

To prepare for this exam, it is highly recommended to review:

- Lecture slides
- Workshop supplementary materials
- Workshop exercises
- Running case study (Tony's Pizza)
- Textbook (See Learning Resources in LMS)

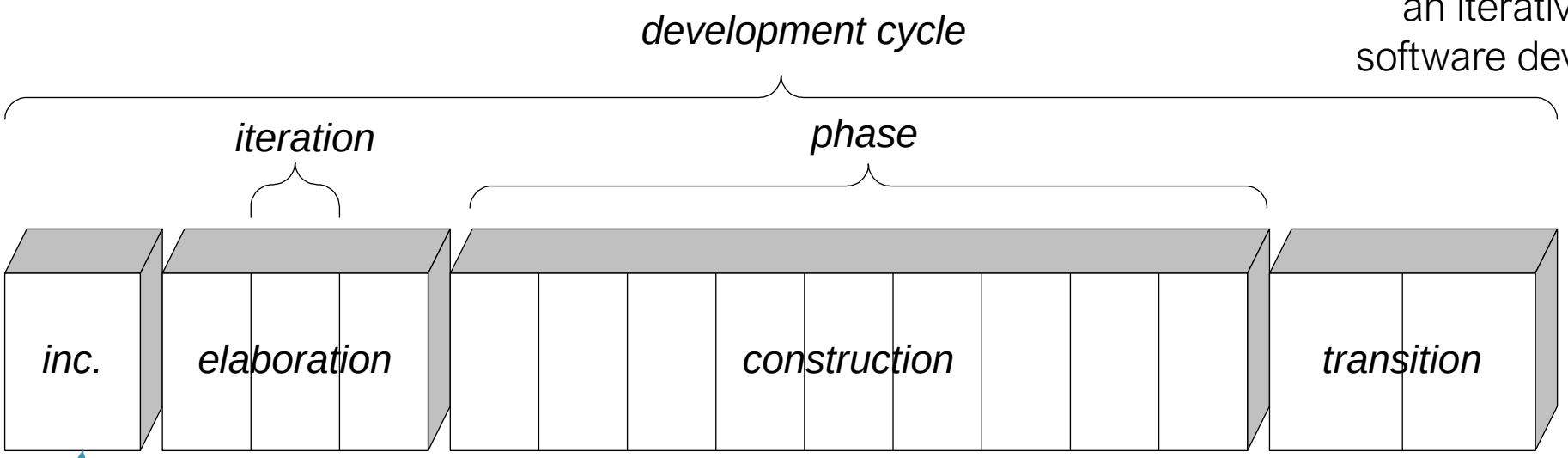
We recommend you study for this exam like you would for any other and aim for a good understanding of the key areas of focus, so you can craft the best answer you can during the exam time.

Subject Aims & Objective

- The **aim** of the subject is to teach you about 'Software Modelling' and 'Software Design'.
- Software Design is all about *purposefully* choosing the **behaviour** and **structure** and of your software system.
  - System behaviour: how your systems responds to inputs and events
  - System structure: how parts of the system collaborate to achieve the goals of the system
- Software Modelling is the creation of tangible, but abstract, representations of a system so that you can communicate your design ideas, critique them and explore viable alternatives

An Overview

Unified Process:
an iterative & incremental
software development process



Inception Phase:
- Analysis of ~10% of use cases
- Go or no Go decision

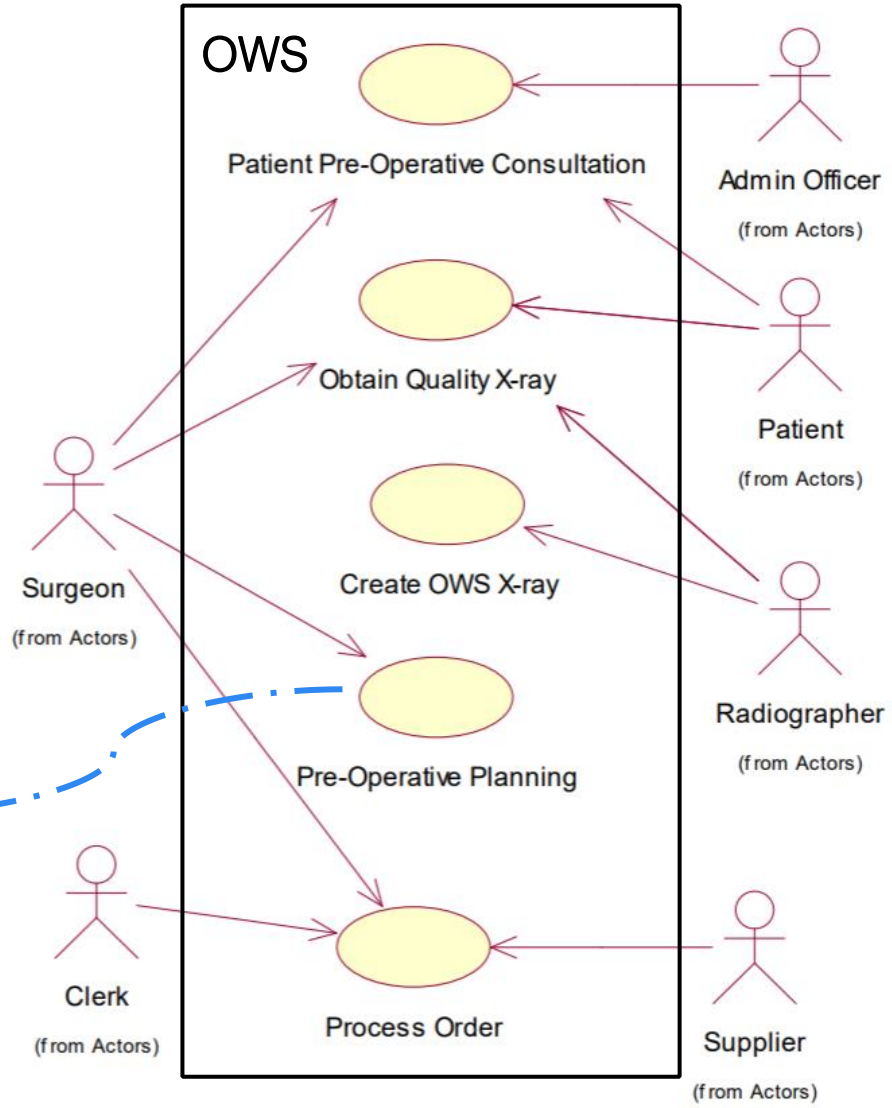
Outcome:
- Common Vision
- Use case model (Use case text + Use case diagram)

Use Case Model

Definition: Use Case is a list of actions or event step that define how the user (or some actors) interact with the system to achieve the goal

- **Use case text** is a written description (a story) of some actors using a system
- **Use case diagram** shows the relationships between actors and use cases in the system

Use Case: Pre-Operative Planning (brief)
The surgeon inspects and annotates an OWS x-ray; judgement is made regarding the femur neck resection lines and location of replacement components. Decisions are also made by the surgeon regarding type and size of replacement components, and the accessories needed for the operation. An operation plan is prepared to reflect decisions made during Pre-Operative Planning. Orders for the components and accessories are generated. A Fad may also be prepared, and some details of the Pre-Operative Plan are added to Patient Details.



An example of a use case diagram

Example: Exam 2018 S1

Q8. Draw a use case diagram, covering the Pennand ATM description above.

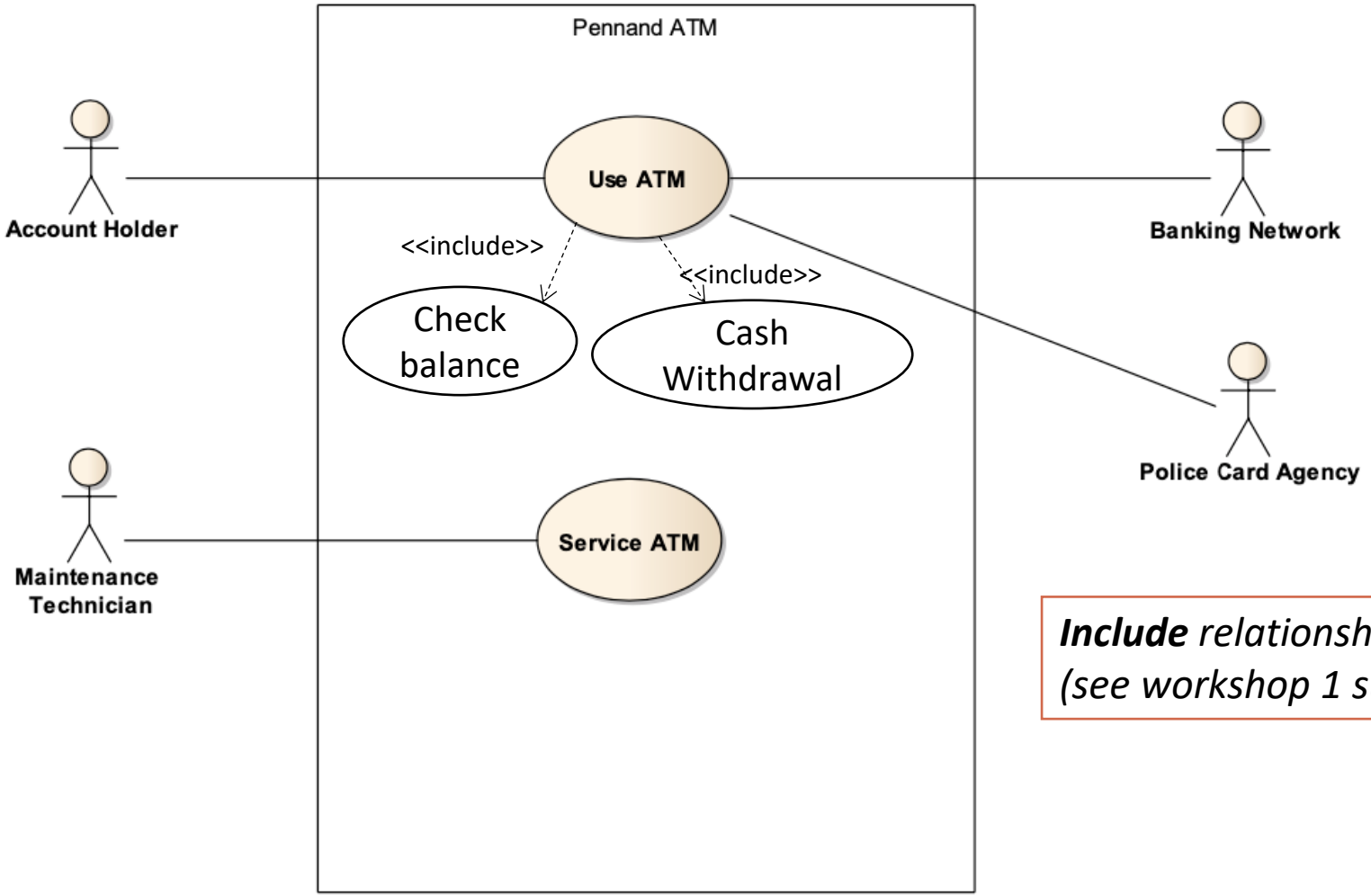
1. The ATM allows an account holder to check the balance or make a withdrawal on one of their accounts.
2. When the ATM is idle, they can insert their transaction card and enter their PIN; the ATM will authenticate (via the banking network) the id from the card and their PIN.

…. <See complete version in LMS or Practice Exam> …

12. When a customer first inserts their card, the ATM will photograph the customer and check the card details against a list of stolen cards (stored locally); a stolen card will be kept in the ATM and a message sent to the Police Card Agency, before the ATM returns to the idle state.
13. If the ATM at any stage detects a fault, it switches itself into out-of-service mode; a maintenance technician can then use this mode to conduct diagnosis/repairs, before shutting down and restarting the ATM.

Example: Exam 2018 S1

Use the Boss Test, EBP, or Size test to justify the level to express use case

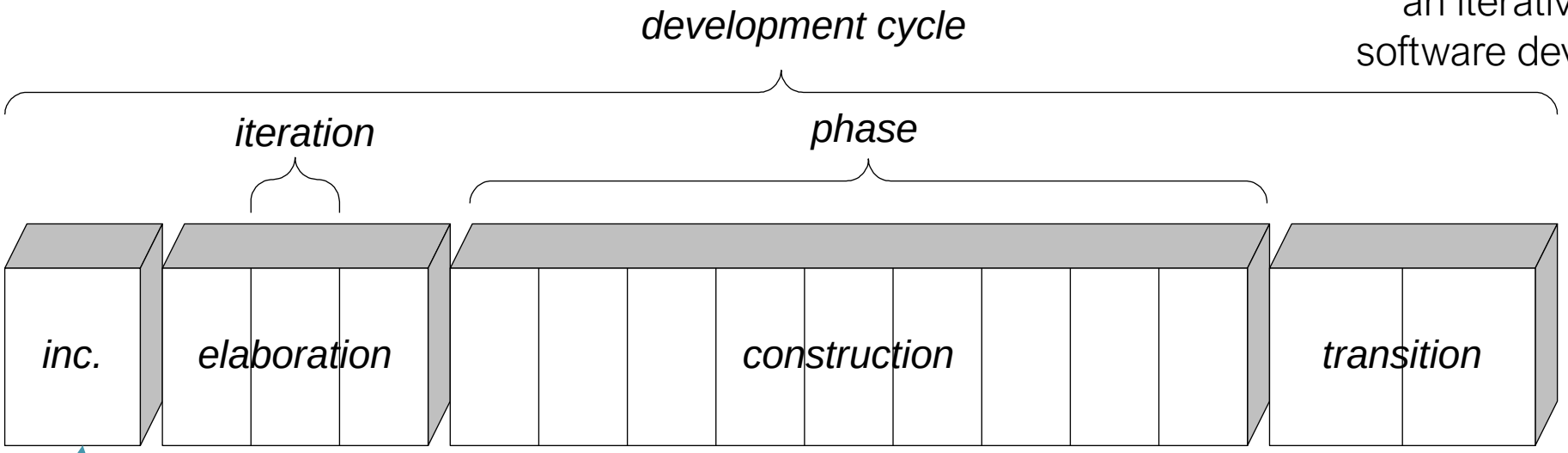


***Include** relationship can be added. (see workshop 1 sup materials)*

Answers for Exam 2018 S1 is available in Sample Exam page under the LMS exam module

An Overview

Unified Process:
an iterative & incremental
software development process



Inception Phase:
- Analysis of ~10% of use cases
- Go or No Go decision

Outcome:
- Common Vision
- Use case model (Use case text + Use case diagram)

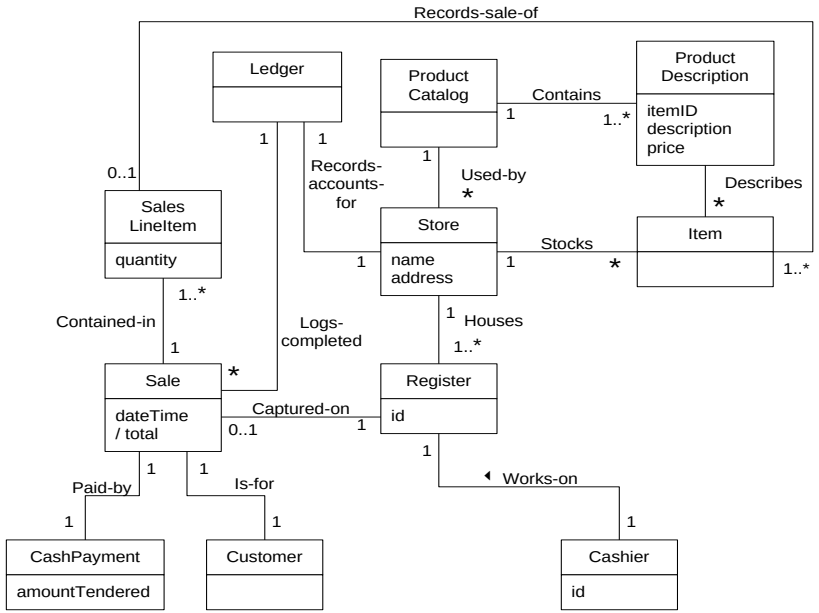
Elaboration Phase:
- Discover and stabilise most of the requirements
- Building the core architecture, analyse the high-risk elements
- Start to think about the design

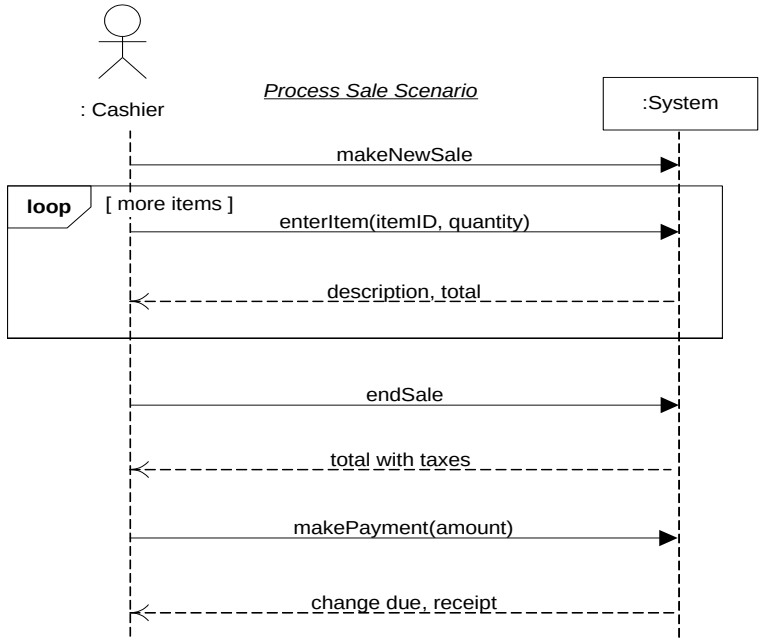
Outcome:
- Domain and Design models for some requirements
- Software Architecture Doc (e.g., component diagram, factor table, technical memo)

Domain Model

Domain models capture the concepts, attributes, and associations in the domain

- **Domain class diagram** provides a *static* context of system: A visual dictionary of noteworthy abstractions, domain vocabulary, and information content of the domain
- **System Sequence diagrams** provides *dynamic* context of system: How actors interact with system (as a black box) and how the system respond
- **Domain State Machine** diagrams can capture *dynamic* behaviour in the domain in terms of states, events, and state transitions
- Domain models should be expressed at the ***abstract level of intention***
  - Provide essential abstractions in information required to understand the domain in the context of the current requirements
  - Aid people in understanding the domain





Architectural Analysis

Definition: An activity to identify *factors* that will influence the architecture, to understand their variability and priority, and resolve them

Architectural Analysis includes identifying and analysing:

- *Architecturally significant requirement*: The requirement that can have a large impact on the design (especially when it was not considered at the beginning)
- Variation points
- Potential evolution points

Goal: To reduce risk of missing a critical factor in the *design* of a system

Architecturally Significant Requirements

Functional Examples	Description
Auditing	Provide audit trails of system execution.
Localization	Provide facilities for supporting multiple human languages.
Printing	Provide facilities for printing.
Workflow	Provide support for moving documents and other work items, including review and approval cycles.

Non-Functional Examples	Description
Usability	Incl. UX, aesthetics, and consistency in the UI.
Reliability	Incl. availability, accuracy of calculations, failure recovery.
Performance	Incl. throughput, response time, recovery time, start-up time.
Supportability	Incl. testability, adaptability, maintainability.

Example: Architecture Factor Table

Reliability—Recoverability of POS

Factor	Recovery from remote service (e.g., Tax Calculator) failure
Measures and quality scenarios	When remote service fails, re-establish connectivity with it within 1 min. of its detected re-availability, under normal store load in a production environment.
Variability (current flexibility and future evolution)	current flexibility - our SME says local client-side simplified services are acceptable (and desirable) until reconnection is possible. evolution - within 2 years, some retailers may be willing to pay for full local replication of remote services (such as the tax calculator). Probability? High.
Impact of factor (and its variability) on stakeholders, architecture and other factors	High impact on large-scale design. Retailers really dislike it when remote services fail, as it prevents them from using a POS to make sales.
Priority for Success	High
Difficulty or Risk	Medium

Example: Technical memo

Technical Memo: Issue: Reliability—Recovery from Remote Service Failure

Factors

Robust recovery from remote service failure, e.g., tax calculator, inventory

Solution

To satisfy the quality scenarios of reconnection with the remote services ASAP, use smart Proxy objects for the services, that on each service call test for remote service reactivation, and redirect to them when possible.

Where possible, offer local implementations of remote services. For example, implementing a small cache to store data (e.g., tax rates)

Achieve protected variation with respect to location of services using an Adapter created in a ServicesFactory.

Example: Technical memo (cont)

Motivation

Retailers really don't want to stop making sales! Therefore, if the NextGen POS offers this level of reliability and recovery, it will be a very attractive product, as none of our competitors provide this capability.

The small product cache is motivated by very limited client-side resources.

The real third-party tax calculator is not replicated on the client primarily because of the higher licensing costs, and configuration efforts (as each calculator installation requires almost weekly adjustments).

This design (Adapter and ServiceFactory) also supports the evolution point of future customers willing and able to permanently replicate services such as the tax calculator to each client terminal.

Unresolved Issues

none

Alternatives Considered

A “gold level” quality of service agreement with remote credit authorization services to improve reliability. It was available, but much too expensive.

Design Model

Design Model is a tangible representation of software classes to illustrate the structure, collaboration, and responsibilities of software objects

- **Design class diagram** provides a static view of the class names, attributes, and method signatures
- **Design sequence diagram** provides a dynamic view of how software objects interact via messages
- **State Machine diagram** is a behaviour model that can capture the dynamic behaviour of an object (or a system) in terms of states, events, and state transitions

To create Design model:

- Use the domain models to **inspire** the design of software objects
  - To reduce representational gap between how stakeholders conceive the domain and its representation in software
- Use a responsibility-driven design: GRASP and GoF design patterrns

GRASP

GRASP concerns assigning responsibility to a software object:

Generic Problem: "Who should be responsible for Task *X?*"

Pattern/Principle	Brief Solution
Creator	If an object A contains or often use an object B, an object A should be responsible for creating an object B (Task X = Creation of B)
Information Expert	If an object A has a relevant information for Task X, an object A should be responsible for Task X
Low Coupling	Assign the responsibility for Task X to an object to make the coupling of the system remains low
High Cohesion	Assign the responsibility for Task X to an object that has strongly related responsibilities (the cohesion of the system remains high)
Pure Fabrication	If the solutions offered by other patterns (e.g., Creator, Expert) violate Low Coupling and High Cohesion, assign the responsibility for Task X to an artificial class that was not represented in the domain model

GRASP

GRASP concerns assigning responsibility to a software object:

"Who should be responsible for Task *X?*"

Pattern/Principle	Brief Solution
Indirection	To avoid direct coupling, an *intermediate* object to mediate between other components or services should be responsible for Task X.
Controller	What first object beyond the UI layer receives and coordinates ("controls") a system operation? -> Assign responsibility to a class representing one of: (a) a "root" object; or (b) a use case scenario that deals with the event. Note: • *Motivated by Indirection, Controller helps to avoid direct coupling between app logic and UI levels* • *Controller can be an artificial class motivated by Pure Fabrication*
Protected Variation	If there are variations or instability in these elements, assign responsibilities to create a stable interface
Polymorphism	If the behaviour of Task X is varied based on type or domain classes have various forms, assign responsibility for the behavior—using polymorphic operations—to the types for which the behavior varies

GoF Design Patterns

GoF Design Patterns: A set of patterns for specific problems

Pattern	Brief Description
Façade	**Q**: How to design a common unified interface for a disparate set of implementations or interfaces such as within a subsystem? **A:** Create a Pure Fabrication object called a Façade object to define a single point of contact to the subsystem
Decorator	**Q**: How to dynamically add behaviour to individual objects at run-time without changing the interface presented to the client? **A**: Encapsulating the original concrete object inside an abstract wrapper interface. Then, let the *decorators* that contain the dynamic behaviours also inherit from this abstract interface.
Builder	**Q:** How can a class that include creating a complex object be simplified? **A:** Encapsulating creating and assembling the parts of a complex object in a separate *Builder* object

GoF Design Patterns

GoF Design Patterns: A set of patterns for specific problems

Pattern	Brief Description
Adapter	**Q**: How to provide a stable interface to similar components with different interfaces? **A**: Convert the original interface of a component into another interface, through an intermediate adapter object.
Composite	**Q**: How to treat a group or composition structure of objects the same way (polymorphically) as a non- composite (atomic) object? **A**: Define classes for composite and atomic objects so that they implement the same interface.
Factory	**Q**: Who should be responsible for creating objects when there are special consideration? **A**: Create a Pure Fabrication object called a Factory that handles the creation

GoF Design Patterns

GoF Design Patterns: A set of patterns for specific problems

Pattern	Brief Description
Singleton	**Q:** Exactly one instance of a class is allowed—it is a "singleton." Objects need a global and single point of access **A:** Define a static method of the class that returns the singleton
Strategy	**Q**: How to design for varying, but related, algorithms, strategies or policies? **A**: Define each algorithm/policy/strategy in a separate class, with a common interface.
Observer	**Q**: Different kinds of subscriber objects are interested in the state changes or events of a publisher object and want to react in their own unique way when the publisher generates an event. **A**: Define a "subscriber" or "listener" interface. Subscribers implement this interface. The publisher can dynamically register subscribers who are interested in an event and notify them when an event occurs.

Exam MCQ: Design Patterns

Which of the following statements accurately describes the Singleton design pattern?

A. The Singleton pattern is not a design pattern but rather a programming language feature.

B. The Singleton pattern ensures that only one instance of a class can be created.

C. The Singleton pattern allows multiple instances of a class to be created.

D. The Singleton pattern is used to implement inheritance in object-oriented programming.

Exam MCQ: Design Patterns

Which GRASP pattern promotes high cohesion and low coupling by assigning responsibilities for handling a particular set of user inputs to a single class?

A. **Controller**
B. Creator
C. Low Coupling
D. High Cohesion

Exam MCQ: Design Patterns

You are developing a software application for an e-commerce platform. The application needs to create complex product objects with various optional features such as colour, size, and additional accessories. The product objects should be created in a step-wise manner with which pattern?

A. Singleton

B. Observer

C. Decorator

D. Builder

Exam MCQ: Design Patterns

In software design, which of the following GRASP principles helps you identify the class which should be responsible for creating objects?

A. **Creator**
B. Controller
C. Low Coupling
D. High Cohesion

Exam MCQ: Design Patterns

AutoBike Co. Ltd. is developing a bike assembly system. The assembly process follows a series of defined steps starting with the frame and ending with the seat. However, there are variations in some of these steps based on the specific bike model . Which pattern should this company use to help with the assembly system design?

A. Pure Fabrication

B. Controller

C. Decorator

D. Builder

Exam Information

Exam Information/Instructions

Exam Date/Venue:

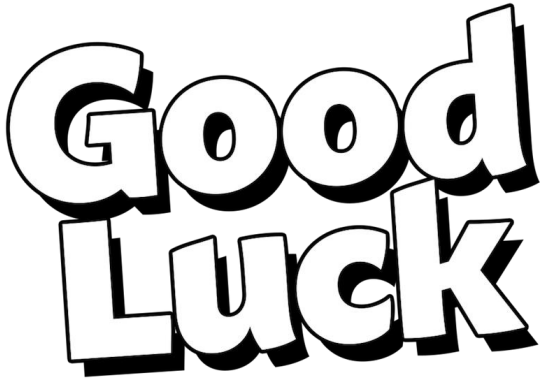
For the Main and AEA exams, please refer to the information provided by the University.

Exam Consultation: There will be drop-in exam consultation sessions (via Zoom) prior to the exam. Keep an eye out for an announcement for date and time. We will also continue to answer questions on Ed Discussion.

Important Note:

- On-campus, paper-based exam
- Two double-sided A4 pages of notes can be taken into the exam
- Practice/previous exams with similar format are available in LMS under Exam > Sample Exams
- Exam questions include multiple-choice questions, short-/long- answers
- Some questions will require you to draw a diagram
- All diagrams and answers must be *legibly drawn or written*

Exam Principles



- Is there reading or writing code in Exam?
  - *Short Answer*: Reading code -> Possibly, yes; Writing code -> Possibly, but snippets only
  - *Long Answer*: Coding is not the core focus of this subject, but coding can demonstrate the effectiveness of design. You should be able to recognize/produce code structures relating to design patterns/principles.

- What kind of questions/topics will be in the exam?
  - *Short Answer*: All key topics mentioned today can be in the exam
  - *Long Answer*: Based on the learning objective of this subject, you should be able to "Apply the knowledge of modelling and design to solve computing problems" and "Evaluate and apply appropriate choices of software tools in the modelling and design process". These are the key focus of the exam. And you should demonstrate these in your exam answers.

Q&A about exam

Q: Are the workshop supplementary materials relevant to the exam?

A: Yes. However, the topics that will be in the exam will mainly be in the lecture slides and workshop exercises.

Q: Will TDD and refactoring be in the exam?

A: Possibly because TDD and refactoring are part of workshop supp materials.

Q: Do we need to know the specific tools to use, e.g., Amazon web service?

A: No. That’s not the key focus of this subject

Q: Would there be a detailed case study in the exam?

A: There could be a case study in the exam, but not as big as the project assignments. The size and complexity would be reasonable given the fixed timeframe (2 hours of exam).

Q&A (Others)

Q: How detailed should our design sequence diagram be?

A: It should reflect how the code works. If your diagram is too large, you can use ref frame to break the diagram into parts.

Q: What kind of classes should and shouldn't be in the domain model?

A: The domain model capture the understanding about the domain and requirements. Classes should be meaningful to the stakeholder, relevant to the problem being solved, and not relate to an implementation to be in the domain model.

Q: A class made for pure fabrication wouldn't be included in the domain model, right?

A: Correct.

Q: Should architecture always go first, before the domain and software design?

A: Architecture, Domain model, and Design model can be developed concurrently in multiple iterations. It is not like a waterfall model where you have a solid architecture before doing something else.

Lecture Identification

Lecturer: Philip Dart

Semester 2, 2024

© University of Melbourne 2024

These slides include materials from:

Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition, by Craig Larman, Pearson Education Inc., 2005.